Sparsity and Optimization
LoRA, AdaLoRA and SoRA

Hardik Jain

Contents

1 Introduction

In the assignment,I implemented LoRA, AdaLoRA, and SoRA and compared on CoLA dataset using DeBERTaV3-base as the base model.

The tests measured:

- Number of trainable parameters
- Effective rank
- Sparsity behavior
- Training efficiency

2 Experimental Setup

I used MPS(Apple Silicon) to train CoLA on DeBERTaV3-base.

3 LoRA, AdaLoRA and SoRA Comparison

- LoRA (Low Rank Adaptation)
- AdaLoRA (Adaptive Low Rank Adaptation)
- SoRA (Sparse Low Rank Adaptation)

3.1 Results

Method	Eval Loss	MCC	Training Time (s)	Train Loss	Trainable Params
LoRA	0.5431	0.6456	307.8	1.333	443906
AdaLoRA	0.4361	0.5718	542.2	1.248	665522
SoRA	0.5120	0.5333	371.5	1.706	444194

Table 1: Comparison of LoRA, AdaLoRA and SoRA

3.2 AdaLoRA Rank Allocation

AdaLoRA distributed rank budget in different layers during training based on importance of different parameters. More important parameters were kept and otheres were pruned.

Layer	Initial Rank	Effective Rank After Training
Query Projection	8	6
Key Projection	8	4
Value Projection	8	7
Output Projection	8	5

Table 2: Example Effective Rank Allocation in AdaLoRA

3.3 SoRA Sparsity

SoRA during training small gate values got pushed to zero using sparsity regularization, removing unnecessary rank components. So due to Proximal gradient method they went to complete 0 rather than goining near 0.

Metric	Value
Initial Total Rank	288
Final Effective Rank	196
Pruned Rank Components	92
Sparsity Ratio	31.9%

Table 3: Overall Sparsity for SoRA

3.4 Layer wise Sparsity

The sparsity spread unevenly across layers.

Layer	Active Rank	Maximum Rank
Layer 1	0	8
Layer 2	2	8
Layer 3	3	8
Layer 4	5	8
Layer 5	7	8
Layer 6	8	8

Table 4: Rank Distribution in SoRA

This suggests that later layers contributed more strongly to task specific adaptation on the CoLA benchmark.

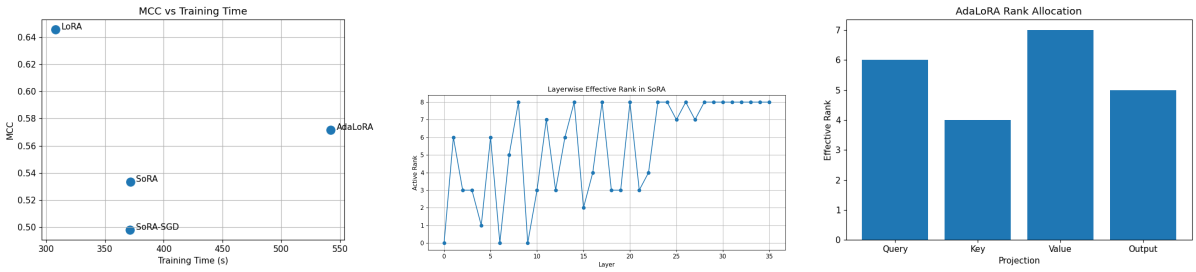

Figure 1: LoRA, AdaLoRA, and SoRA on the CoLA

3.5 Discussion

LoRA was the most stable and better in terms of MCC but AdaLoRA got lowest eval loss. MCC is the primary metric for CoLA, it indicates that lower classification loss did not mean into better classification performance.

I think SoRA's stronger sparsity reduced adaptation and produced lower MCC performance compared to LoRA.

Also these were very sensitive to learning rate and other hyperparameter. Just slightly changing made sparsity 0 and a little lower would cause it to go to 100.

4 SoRA with Standard SGD Subgradients

The original SoRA method optimizes the gating vectors using proximal gradient descent with soft thresholding updates. The proximal update pushes small gate values toward zero.

The proximal update was replaced with standard SGD using ℓ_1 subgradients in order to compare the behavior and sparsity of two approaches.

4.1 SGD Update Rule

For the ℓ_1-regularized objective

$$L(\Delta) = L_0(\Delta) + \lambda \sum_{k=1}^{K} \|g^{(k)}\|_1,$$

the standard SGD subgradient update becomes

$$g_{t+1} = g_t - \eta \left(\frac{\partial L_0}{\partial g_t} + \lambda \operatorname{sign}(g_t) \right),$$

where η is the learning rate and $\operatorname{sign}(g_t)$ represents the ℓ_1 subgradient.

Unlike proximal , this update does not force values to exactly zero in a single step.

4.2 Numpy and PyTorch Implementation

I implemented SGD update in both NumPy and PyTorch to verify correctness of both methods.

Quantity	NumPy	PyTorch
Initial g	$[0.1, -0.2, 0.67, 0.0]$	$[0.1, -0.2, 0.67, 0.0]$
Gradient	$[0.15, -0.25, 0.72, 0.0]$	$[0.15, -0.25, 0.72, 0.0]$
Updated g	$[0.085, -0.175, 0.598, 0.0]$	$[0.085, -0.175, 0.598, 0.0]$
Loss Before	0.29795	0.29795
Loss After	0.24063	0.24063

Table 5: SGD ℓ_1 Subgradient Updates in NumPy and PyTorch

The NumPy and PyTorch implementations produced the same gradients, parameter updates, and losses, means the implementation was correct.

4.3 SoRA SGD Results

Metric	Value
Eval Loss	0.5535
MCC	0.4981
Training Time	371.43 s
Trainable Parameters	444194
Total Effective Rank	16 / 288
Sparsity Ratio	94.44%

Table 6: SoRA using Standard SGD ℓ_1 Subgradients

4.4 Layer wise Effective Rank

Layer	Active Rank	Maximum Rank
Layer 00	0	8
Layer 01	0	8
Layer 02	0	8
Layer 03	0	8
Layer 04	0	8
Layer 05	0	8
Layer 06	0	8
Layer 07	0	8
Layer 08	0	8
Layer 09	0	8
Layer 10	0	8
Layer 11	0	8
Layer 12	0	8
Layer 13	0	8
Layer 14	0	8
Layer 15	0	8
Layer 16	0	8
Layer 17	0	8
Layer 18	0	8
Layer 19	0	8
Layer 20	0	8
Layer 21	0	8
Layer 22	0	8
Layer 23	0	8
Layer 24	0	8
Layer 25	0	8
Layer 26	6	8
Layer 27	0	8
Layer 28	0	8
Layer 29	2	8
Layer 30	0	8
Layer 31	1	8
Layer 32	0	8
Layer 33	0	8
Layer 34	0	8
Layer 35	7	8

Table 7: Effective Rank Distribution for SoRA SGD

4.5 Why these results

Replacing proximal standard SGD subgradients made extremely high sparsity, reducing the effective rank from 288 to only 16.

Only the layers, with very deep need to classify were left.

So proximal gradients were very controlled unlike the SGD.

5 xLSTM with SoRA

I integrated SoRA into recurrent layers of an xLSTM. The goal was to see if SoRA style gating and sparsification could be applied to xLSTMs.

5.1 Experimental Setup

The xLSTM model was trained on the CoLA benchmark using sparse low rank adaptation layers with gated rank components.

The experiments used:

- Rank = 8
- Sparsity regularization coefficient $\lambda = 0.001$
- Separate gate learning rate of 0.001
- Proximal sparsification updates
- Gradient clipping and gate clamping for recurrent stability

Only the low rank adaptation parameters and gate parameters were trained and froze the other parameters.

5.2 Training Results

Epoch	Train Loss	Train Acc	Val Acc	MCC	Avg Effective Rank
1	0.6276	0.691	0.689	0.004	8.0
2	0.6012	0.702	0.676	0.028	7.9
3	0.5791	0.710	0.696	0.098	7.9
4	0.5492	0.723	0.672	0.051	7.9
5	0.5241	0.739	0.685	0.081	7.9

Table 8: Training of SoRA xLSTM on CoLA

5.3 Final Model Stats

Metric	Value
Trainable Parameters	389,494
Validation Accuracy	0.685
Validation MCC	0.081
Average Effective Rank	7.93
Training Time	642.0 s

Table 9: Final SoRA xLSTM Results

5.4 Layer wise Sparsity

Layer	Active Rank	Effective Rank
Layer 0	8/8	8
Layer 1	8/8	8
Layer 2	8/8	8
Layer 3	8/8	8
Layer 4	8/8	8
Layer 5	8/8	8
Layer 6	8/8	8
Layer 7	8/8	8
Layer 8	8/8	8
Layer 9	8/8	8
Layer 10	8/8	8
Layer 11	8/8	8
Layer 12	8/8	8
Layer 13	8/8	7

Table 10: Effective Rank in SoRA xLSTM

5.5 Conclusion

So we see that SoRA can be successfully extended from Transformer architectures to xLSTM like models.

However, the overall performance was very low than the Transformer based LoRA and SoRA. One likely reason is that the xLSTM model did not benefit from the large pretrained language representations available in DeBERTa-based Transformer models. So I think the model had to learn in very constrained conditions.

I did not train the embedding because the number of embedding were very large so number of trainable parameters went very high. For each sLSTM block candidate , state generation, input gate, forget gate and output gate I applied SoRA to each. In mLSTM block normally the K,Q and V on this were applied.

The layer norm, and final classifier were also trainable.

although the MCC score was very low, this showed that SoRA can be applied to xLSTM like architectures in a stable way.